# COSA, WP2: CoE on SDC

Centre of Excellence on statistical COnfidentiality, Seasonal Adjustment and time series

# COSA

- ‘Continuation’ of the STACE project
- WP2 is on Statistical Disclosure Control
- Some “old”, some “new”(*) partners:
  - Netherlands, Austria, France, Germany, Iceland, Norway (*) and Slovenia.

# WP2: CoE on SDC

- Maintain the UGSDC and its meetings
- Maintain github sources repositories
- Produce vision document on modernization of τ-argus
- At least one PoC evaluating ideas of ‘future proof τ-argus’
- Extend guidelines Output Checking

# T-argus modernization: why?

- T-argus is widely used software to protect tabulated data
- Development of τ-argus started in 1990's
- Still under development and maintenance
- Long history => complicated structure/architecture

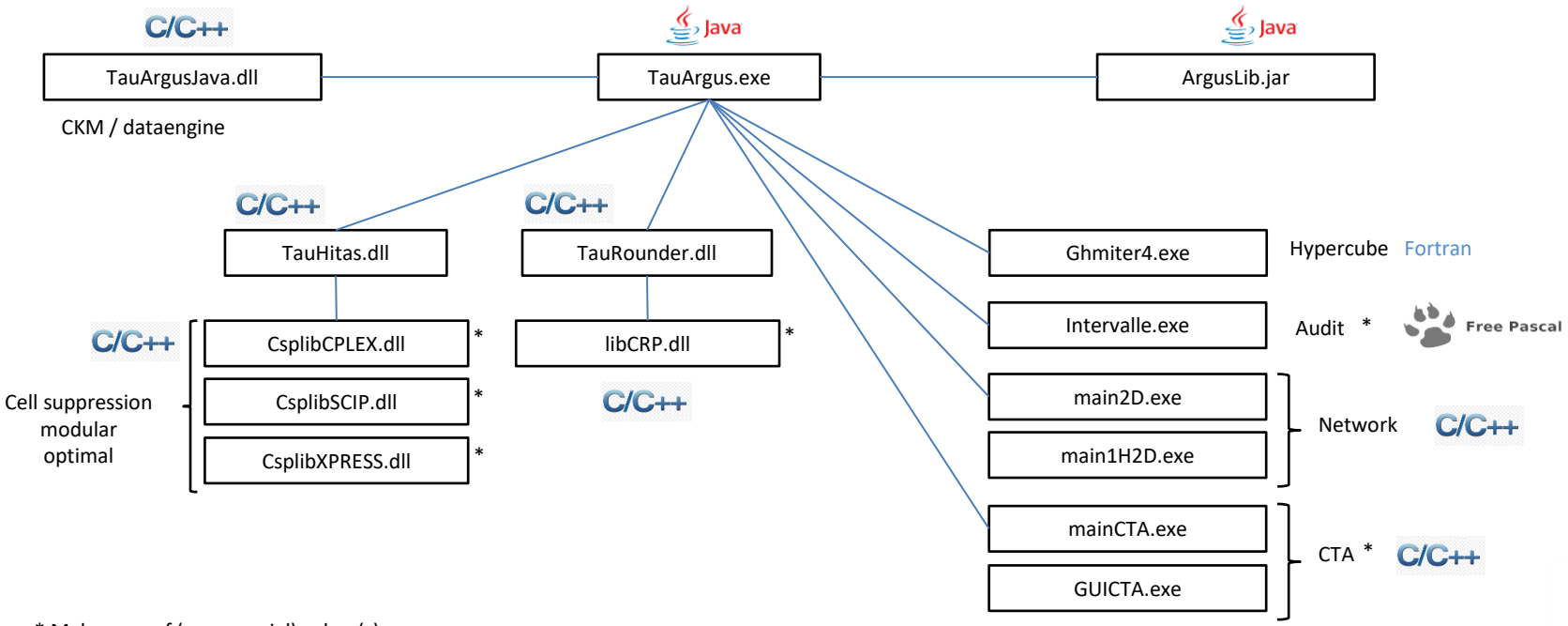
# T-argus modernization: why?

τ-ARGUS

Started development of SDC software in the nineties of the 20th century

- Within Statistics Netherlands
- In European context, with (partial) funding by Eurostat
- Internatially used (‘standard’)
- Still under development and maintenance
- Long history => complicated structure/architecture

# T-argus modernization: why?




* Makes use of (commercial) solver(s)

# T-argus modernization: why?

- Stand alone software (Java GUI)
- Batch/script
- R ‘wrapper(s)’ `rtauargus` and `sdcTable`
- As a service?
- Included in production process
- …

# T-argus modernization: why?

- Hard to maintain/extend
- Hard to make alternative GUIs
- Different programming languages
- SPOKs for certain methods/implementations
- Some developers no longer available
- Open Source, but not many new developers…
- …

# T-argus modernization: what?

Long term (beyond COSA)

- Completely seperated frontend / backend
- Interface between frontend and backend
  - Independent of frontend programming language
- Backend + interface available, separated from frontend
- Backend platform independent
- Backend using one programming language (C/C++)
- Backend facilitates use of different solvers
- Backend easily augmented with new methods
- Interface easily used by different frontend implementations

# T-argus modernization: what?

Short term (within COSA)

- Vision document
  - Decide on architecture
  - Decide on interface
  - Decide on backend functionality
  - PoCs
    - τ-argus Light (use of interface/frontend)
    - GaussSuppression (add new method to backend)

# Scenario(s)

Scenario I: Developing protection protocol

- Intended for new table/statistic
- Iterative process
  - Different design
  - Different methods / parameters
- Visually/manually review results

# Scenario(s)

Scenario II: Applying protection protocol

- Intended for existing table/statistic
- Predefined design
- Predetermined methods / parameters
- (Visually/manually review results)

# Workflow I: microdata input

- Read microdata + metadata (.asc + .rda + .hrc)
- Specify table structure
  - Spanning variable(s)
  - Response variable
- Specify sensitivity measure(s)
- Aggregate microdata (tabulation)
- Identify sensitive cells

# Workflow I: microdata input

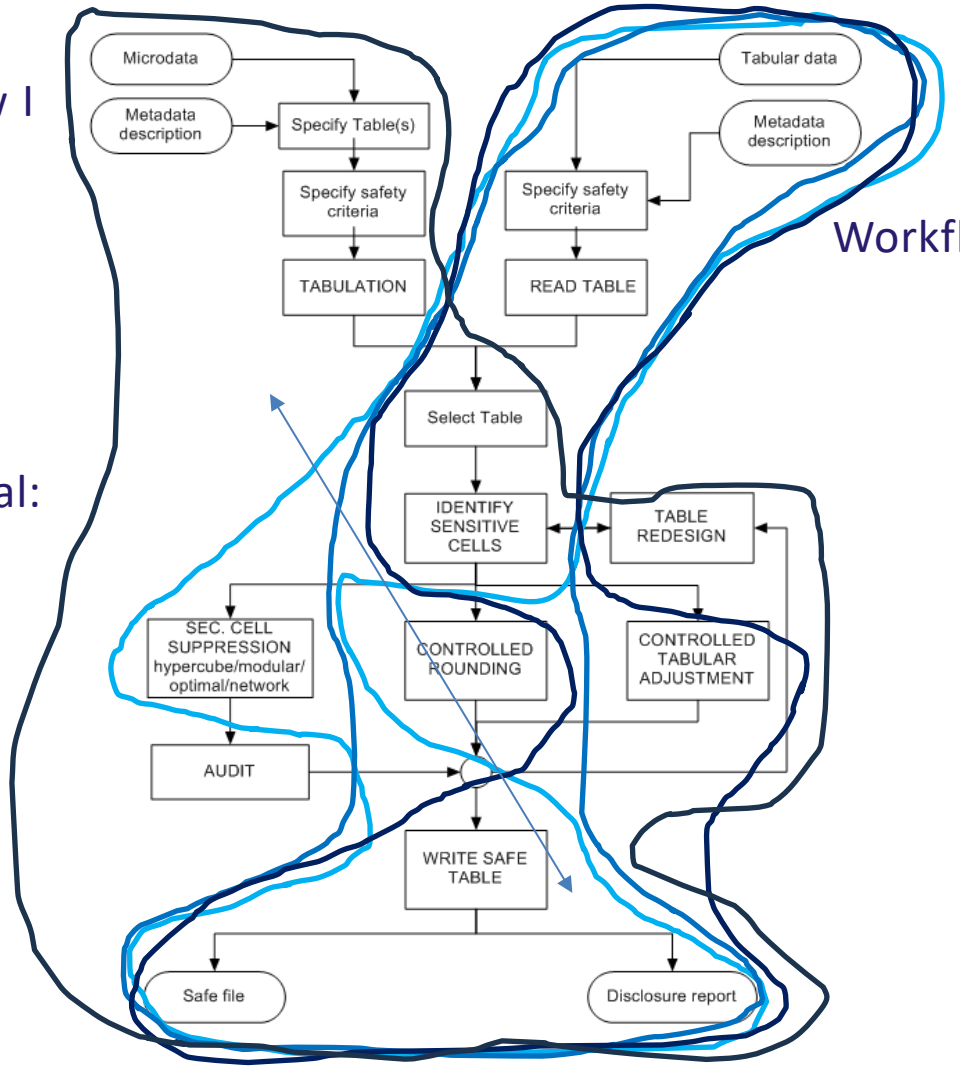
- Apply protection method
  - Redesign
  - Suppression (Hypercube, Modular, Optimal, Network)
  - Rounding (CRP)
  - Adding noise (CKM, CTA)
- Saving (protected) table + producing report file

# Workflow II: tabular input

- Read table + metadata (.tab + .rda + .hrc)
- Correct for additivity
- Identify sensitive cells
  - User defined, sensitivity rules
- Apply protection method
  - Suppression (Hypercube, Modular, Optimal, Network)
  - Rounding (CRP)
  - Adding noise (CKM, CTA)
- Saving (protected) table + producing report file

Workflow I

Uit τ-argus manual:




Workflow II

# Workflow(s)

Additional actions (optional):

- Create apriori-file
- Read and process apriori-file
- Audit solution

# Scenarios and workflows

Any additional scenarios and/or workflows?

What are you missing w.r.t. your use of τ-argus?

# Ideas: library

- Library with SDC functionality
- Written in C/C++ with C interface
- Decoupled functions with uniform interface
- Implements platform-independent functionality
- Platform-dependent functionality by injection
- Optimization solver by injection

# Ideas: applications

- Use library to implement SDC workflows
- User applications can be written in most languages
- Wrappers can be written for use in Python or R
- Service layer can be written to serve client applications